Quickstart Guide: zztab2fig v0.2.1

Ronald G. Thomas

This quickstart guide covers the key features in zztab2fig v0.2.1. For complete documentation, see the "Object Types and Themes" and "Advanced Features" vignettes.

Installation

```
# Install from GitHub
# devtools::install_github("rgt47/zztab2fig")
```

```
library(zztab2fig)
```

Basic Usage

Convert a data frame to a cropped PDF table:

```
t2f(mtcars[1:6, 1:4], filename = "basic_table", sub_dir = output_dir)
```

	mpg	cyl	disp	hp
Mazda RX4	21.0	6	160	110
Mazda RX4 Wag	21.0	6	160	110
Datsun 710	22.8	4	108	93
Hornet 4 Drive	21.4	6	258	110
Hornet Sportabout	18.7	8	360	175
Valiant	18.1	6	225	105

This generates three files:

- `basic_table.tex` - LaTeX source
- `basic_table.pdf` - Cropped PDF (primary output for document inclusion)
- `basic_table_full.pdf` - Full PDF (before cropping)

Interactive Terminal Graphics (zzvim-R)

When working in a kitty-compatible terminal (Kitty, Ghostty, WezTerm, iTerm2) with zzvim-R, you can preview tables directly in the terminal using `show_table()`:

```
# Display table in terminal with inline preview
show_table(mtcars[1:6, 1:4])

# With theme
show_table(mtcars[1:6, 1:4], theme = "nejm")

# Statistical objects work too
```

```r
model <- lm(mpg ~ cyl + hp, data = mtcars)
show_table(model)
```

Navigate through your table history:

```r
show_table(mtcars[1:6, 1:4])
show_table(iris[1:6, ])
show_prev()      # Go back to mtcars table
show_next()      # Forward to iris table
show_history()   # List all graphics (plots and tables)
show()           # Redisplay current
```

Tables and plots share a unified history. Use `show_history("table")` to filter to tables only, or `show_history("plot")` for plots.

Export options:

```r
show_table(model, theme = "nejm")
show_zoom()              # Open PDF in external viewer
show_save("table.pdf")   # Save current as PDF
show_save("table.png")   # Save current as PNG
```

Embedding Tables in Rmd/Quarto

For reproducible documents, use `t2f()` with `graph_include()`:

```r
# In an Rmd/qmd code chunk:
t2f(model, theme = "nejm") |> graph_include()
```

The `t2f()` function returns the path to the generated PDF, which `graph_include()` passes to `knitr::include_graphics()`.

This is the recommended workflow for batch rendering:

1. **Interactive exploration**: Use `show_table()` to preview and compare tables
2. **Final document**: Use `t2f() |> graph_include()` for reproducible embedding

```r
# Interactive session (terminal)
show_table(model, theme = "nejm")
show_table(model, theme = "apa")
show_prev()  # compare themes

# Rmd chunk (batch mode)
t2f(model, theme = "nejm", caption = "Table 1") |> graph_include()
```

Journal Themes

Built-in themes provide journal-specific styling:

```r
t2f(mtcars[1:6, 1:4], theme = "nejm",
    filename = "nejm_table", sub_dir = output_dir)
```

	mpg	cyl	disp	hp
Mazda RX4	21.0	6	160	110
Mazda RX4 Wag	21.0	6	160	110
Datsun 710	22.8	4	108	93
Hornet 4 Drive	21.4	6	258	110
Hornet Sportabout	18.7	8	360	175
Valiant	18.1	6	225	105

Available themes: `"nejm"`, `"apa"`, `"nature"`, `"minimal"`

Set a global theme:

```
t2f_theme_set("nejm")
t2f(mtcars[1:6, 1:4])
t2f(iris[1:6, ])
```

Statistical Objects

t2f works directly with statistical objects:

```
model <- lm(mpg ~ cyl + hp + wt, data = mtcars)
t2f(model, filename = "regression", sub_dir = output_dir,
    include = c("estimate", "std.error", "p.value"))
```

Term	Estimate	Std_Error	p_value
(Intercept)	38.752	1.787	<0.001
cyl	-0.942	0.551	0.098
hp	-0.018	0.012	0.140
wt	-3.167	0.741	<0.001

Supported object types include `lm`, `glm`, `anova`, `aov`, `htest`, `matrix`, `table`, plus rms objects (`ols`, `lrm`, `cph`, `orm`, `Glm`, `psm`) and additional types via broom integration (coxph, survreg, nls, polr, prcomp, kmeans, lmerMod, and more).

Inline Tables for R Markdown

Insert tables directly in R Markdown without LaTeX float positioning:

```
model <- lm(mpg ~ cyl + hp, data = mtcars)
t2f_inline(model,
           width = "3in",
           align = "left",
           caption = "Model Results",
           label = "tab:model",
           caption_position = "above")
```

Or use the convenience function for coefficient tables:

```
t2f_coef(model, caption = "Coefficients")
```

Add visual styling with frames and backgrounds:

```r
t2f_inline(model,
           width = "3in",
           frame = TRUE,
           frame_color = "gray",
           background = "gray!5",
           inner_sep = "4pt")
```

Note: Add `\usepackage{caption}` to your R Markdown YAML for captions.

Text Wrapping Around Tables

To wrap paragraph text around a table, use the `wrapfig` LaTeX package. First, add `\usepackage{wrapfig}` to your YAML header, then use inline R code with `knitr::asis_output()`:

```r
wrap_table <- function(filename, position = "r", width = "0.45\\textwidth") {
  pdf_path <- paste0(filename, "_cropped.pdf")
  knitr::asis_output(sprintf("\\begin{wrapfigure}{%s}{%s}
  \\centering
  \\includegraphics{%s}
\\end{wrapfigure}", position, width, pdf_path))
}
```

Then embed the table inline using backtick-r syntax. The following paragraph demonstrates text wrapping:

This text flows around the table on the right. The position parameter controls placement: "r" for right, "l" for left. The width parameter sets how much horizontal space the table occupies. Text continues to wrap naturally until sufficient content fills the space beside the table. This approach works well for small summary tables that complement the narrative without interrupting the reading flow. Notice how the paragraph continues alongside the table, making efficient use of page space. When the text extends beyond the table height, it returns to full width automatically. This technique is particularly useful in research papers where you want to discuss results while displaying the supporting data in close proximity to the discussion. Additional text can continue here to demonstrate that normal formatting resumes after the wrapped section.

	mpg	cyl	disp
Mazda RX4	21.0	6	160
Mazda RX4 Wag	21.0	6	160
Datsun 710	22.8	4	108
Hornet 4 Drive	21.4	6	258

Model Comparison

Compare multiple models side-by-side:

```r
m1 <- lm(mpg ~ cyl, data = mtcars)
m2 <- lm(mpg ~ cyl + hp, data = mtcars)
m3 <- lm(mpg ~ cyl + hp + wt, data = mtcars)

t2f_regression(
  Model1 = m1, Model2 = m2, Model3 = m3,
  stars = TRUE,
  filename = "model_comparison",
  sub_dir = output_dir
)
```

Term	Model1	Model2	Model3
(Intercept)	37.885* (2.074)	36.908* (2.191)	38.752* (1.787)
cyl	-2.876* (0.322)	-2.265* (0.576)	-0.942 (0.551)
hp		-0.019 (0.015)	-0.018 (0.012)
wt			-3.167* (0.741)
N	32	32	32
R-squared	0.726	0.741	0.843
Adj. R-squared	0.717	0.723	0.826

RMS Package Integration

Full support for Frank Harrell's rms (Regression Modeling Strategies) package, commonly used in biostatistics and clinical trials:

```r
library(rms)
dd <- datadist(mtcars)
options(datadist = "dd")
```

Coefficient Tables with Odds/Hazard Ratios

```r
model <- lrm(am ~ mpg + wt, data = mtcars)
t2f(model, exponentiate = TRUE, filename = "lrm_odds_ratios")
```

ANOVA-Style Chunk Tests

Useful for models with spline terms:

```r
model <- ols(mpg ~ rcs(wt, 4) + cyl, data = mtcars)
t2f(model, output = "anova", filename = "ols_anova")
```

RMS Model Comparison

```r
m1 <- ols(mpg ~ wt, data = mtcars)
m2 <- ols(mpg ~ wt + cyl, data = mtcars)
t2f_rms_compare(Simple = m1, Full = m2, filename = "rms_comparison")
```

RMS methods: `t2f.ols()`, `t2f.lrm()`, `t2f.cph()`, `t2f.orm()`, `t2f.Glm()`, `t2f.psm()`, and `t2f_rms_compare()`.

Extensibility: Any Broom Object

For any object supported by the broom package, use `t2f_tidy()`:

```r
library(survival)
cox_model <- coxph(Surv(time, status) ~ age + sex, data = lung)
t2f_tidy(cox_model, filename = "cox_results")

t2f_tidy(cox_model,
         tidy_args = list(exponentiate = TRUE, conf.int = TRUE),
         filename = "cox_hazard_ratios")
```

This provides an escape hatch for the 100+ object types broom supports.

Captions and Labels

Add LaTeX captions and cross-reference labels:

```
t2f(mtcars[1:6, 1:4],
    caption = "Motor Trend Car Road Tests (1974)",
    label = "tab:mtcars",
    filename = "captioned_table")
```

Column Alignment

Explicit or auto-detected alignment:

```
t2f(iris[1:6, ], align = NULL)
t2f(mtcars[1:6, 1:4], align = c("l", "r", "r", "r"))
t2f(mtcars[1:6, 1:4], align = "c")
```

Decimal Alignment

Use siunitx for decimal-aligned columns:

```
df <- data.frame(
  Item = c("A", "B", "C"),
  Value1 = c(1.5, 123.45, 12.345),
  Value2 = c(0.001, 10.1, 1000.01)
)

t2f(df, filename = "decimal_aligned", sub_dir = output_dir,
    align = list("l", t2f_siunitx(table_format = "3.3"),
                 t2f_siunitx(table_format = "4.2")))
```

Item	Value1	Value2
A	1.500	0.001
B	123.450	10.100
C	12.345	1000.010

Multi-Page Tables

For large tables that span multiple pages:

```
t2f(mtcars,
    longtable = TRUE,
    caption = "Complete mtcars Dataset",
    filename = "long_table")
```

Quick Reference

Key t2f() Parameters

Parameter	Description	Default
`caption`	LaTeX table caption	NULL
`label`	LaTeX label for cross-refs	NULL
`align`	Column alignment (l/c/r)	NULL (auto)

Parameter	Description	Default
`longtable`	Multi-page table support	FALSE
`crop`	Crop PDF margins	TRUE
`crop_margin`	Margin size (pts)	10
`theme`	Theme name or object	NULL

Key t2f_inline() Parameters

Parameter	Description	Default
`width`	Figure width (LaTeX units)	NULL
`align`	left, center, right	"center"
`caption`	Uses \captionof (no float)	NULL
`label`	LaTeX cross-reference label	NULL
`caption_position`	"above" or "below"	"above"
`frame`	Draw border around table	FALSE
`frame_color`	Border color (xcolor syntax)	"black"
`background`	Background color (xcolor syntax)	NULL
`inner_sep`	Padding inside frame	"2pt"
`format`	"auto", "pdf", "png"	"auto"

Core Functions

Function	Purpose
`t2f()`	Convert object to PDF table
`t2f_inline()`	Inline tables for R Markdown
`t2f_coef()`	Quick coefficient tables
`t2f_regression()`	Compare regression models
`t2f_rms_compare()`	Compare rms models
`t2f_tidy()`	Any broom-supported object
`t2f_theme_set()`	Set global theme
`t2f_theme_get()`	Get current theme
`check_latex_deps()`	Check LaTeX dependencies
`ensure_pdfcrop()`	Install pdfcrop via TinyTeX

Terminal Graphics (zzvim-R)

Function	Purpose
`show_table()`	Display table in terminal
`show_plot()`	Display plot in terminal
`show()`	Redisplay last graphic
`show_prev()`	Previous in history
`show_next()`	Next in history
`show_history()`	List all graphics
`show_zoom()`	Open PDF in viewer
`show_save()`	Export to file
`graph_include()`	Embed in Rmd/qmd (batch mode)

S3 Methods (Base)

Object Type	Method
`data.frame`	`t2f.data.frame()`
`matrix`	`t2f.matrix()`
`lm`	`t2f.lm()`
`glm`	`t2f.glm()`
`anova` / `aov`	`t2f.anova()` / `t2f.aov()`
`htest`	`t2f.htest()`

S3 Methods (rms)

Object Type	Method	Notes
`ols`	`t2f.ols()`	Ordinary least squares
`lrm`	`t2f.lrm()`	Logistic regression (OR)
`cph`	`t2f.cph()`	Cox proportional hazards (HR)
`orm`	`t2f.orm()`	Ordinal regression
`Glm`	`t2f.Glm()`	Generalized linear model
`psm`	`t2f.psm()`	Parametric survival

All rms methods support `output = "coef"` (default) or `output = "anova"` for chunk tests, and `exponentiate = TRUE` for odds/hazard ratios.

S3 Methods (via broom)

Object Type	Package
`coxph`, `survreg`, `survfit`	survival
`nls`	stats
`polr`, `multinom`	MASS, nnet
`prcomp`, `kmeans`	stats
`lmerMod`, `glmerMod`	lme4
`lme`	nlme
`Arima`	stats

Use `t2f_tidy()` for any object type supported by broom.

Theme Presets

Function	Style
`t2f_theme_nejm()`	NEJM (medical journals)
`t2f_theme_apa()`	APA (psychology)
`t2f_theme_nature()`	Nature (scientific)
`t2f_theme_minimal()`	Clean, minimal

System Requirements

- R >= 4.0
- LaTeX distribution (TeX Live, MiKTeX, TinyTeX, or MacTeX)

- `pdflatex` and `pdfcrop` executables
- Optional: ImageMagick (for PNG output and terminal graphics)
- Optional: zzvim-R plugin (for `show_table()` and terminal graphics)
- Optional: jsonlite package (for graphics history)

Check your setup:

```
check_latex_deps()
```

If pdfcrop is missing with TinyTeX:

```
ensure_pdfcrop(auto_install = TRUE)
```